Sistema de Gestión Bibliotecaria

Universidad de Madeira

Proyecto Final – Desarrollo de Software II

Explicación del proyecto para principiantes
Octubre 2026

1. El contexto: ¿qué nos pidieron?

La Biblioteca de la Universidad de Madeira necesita un programa de computador para organizar su trabajo diario. Hoy, prestar un libro, saber quién lo tiene, cuándo debe devolverlo o cobrar una multa puede hacerse en papel o en hojas de cálculo. Eso es lento y genera errores.

El proyecto final de la materia *Desarrollo de Software II* consiste en diseñar y construir un **Sistema de Gestión Bibliotecaria (SGB)**: una aplicación web que permita **registrar, administrar y controlar el préstamo de recursos** a tres tipos de usuarios:

- **Estudiantes**, que piden prestados los recursos.
- **Profesores**, que también piden prestados recursos.
- **Bibliotecarios**, que atienden y registran los préstamos, devoluciones, reservas y bajas.

¿Qué maneja la biblioteca?

No solo libros. Tiene seis "colecciones", y cada una tiene sus propias reglas:

Colección	Qué contiene	Días de préstamo	Renovaciones máx.
General	Libros comunes (mín. 20, con 5 copias cada uno)	15 días	5
Reserva	Libros académicos (mín. 20, con 10 copias cada uno)	5 días	0
Referencia	Diccionarios (20, con 3 copias cada uno)	3 días	2
Hemeroteca	Revistas (mín. 15, con 2 copias cada una)	1 día	1
Mapoteca	Mapas digitales (mín. 15), prestados en una USB	1 día (hasta las 9 p. m.)	0
Tecnoteca	Laptops (mín. 5) y tabletas (mín. 2)	7 días	2

Cada recurso puede estar en uno de cinco **estados**:

- **Disponible**: está en la biblioteca.
- **Prestado**: alguien lo tiene.
- **Reservado**: alguien lo apartó.
- **Perdido**: pasaron 120 días sin devolverlo; el sistema lo cambia solo.
- **Dado de baja**: el bibliotecario lo retiró y explicó por qué.

Las reglas del negocio

El sistema debe controlar automáticamente varias situaciones:

- **Préstamos.** Se guarda quién lo pidió, qué recurso, la fecha y hora, cuándo debe devolverlo y qué bibliotecario lo prestó.
- **Reservas.** Solo para libros de la colección general y tabletas. Si un libro está prestado, otra persona puede "hacer fila" para pedirlo. Cuando se devuelve, el primero de la fila tiene 5 días para reclamarlo. Si no lo hace, pierde el turno y pasa al siguiente. Además, quien tiene el libro ya no puede renovarlo si hay alguien esperando.
- **Renovaciones.** Extender el préstamo sin devolver el recurso, hasta el máximo que permite cada colección.
- **Multas.** Si alguien se atrasa, se le cobra según los días hábiles de atraso:
    - Primer día: 1.000 pesos.
    - Del 2.º al 7.º día: 2.500 pesos.
    - Del 8.º día en adelante: se suman 100 pesos por cada día extra.

  Mientras no pague, no puede pedir préstamos ni hacer reservas.

La base de datos debe tener como mínimo 50 estudiantes, 10 profesores y 3 bibliotecarios.

2. Las herramientas que elegimos (y para qué sirve cada una)

Para construir una aplicación web moderna se combinan varias tecnologías. Cada una cumple un papel distinto, como los oficios en una obra de construcción.

React, HTML y CSS: lo que el usuario ve

- **HTML** es el esqueleto de una página web: títulos, botones, tablas, formularios.
- **CSS** es la "pintura y decoración": colores, tamaños, posiciones, tipos de letra.
- **React** es una herramienta de JavaScript para construir páginas interactivas por piezas reutilizables, llamadas *componentes*. Por ejemplo, diseñamos una vez una "tabla" y la usamos para listar libros, usuarios o multas.
- **Vite** es la herramienta que prepara el código de React para que el navegador lo entienda.

Python con FastAPI: el cerebro

Python es un lenguaje de programación muy popular y fácil de leer. **FastAPI** es una herramienta de Python para crear una **API**.

> Una **API** es como el mesero de un restaurante. Tú (la página web) no entras a la cocina: le haces un pedido al mesero ("dame la lista de libros disponibles"), él va a la cocina y te trae la respuesta. Aquí es donde viven todas las reglas: si se puede prestar, cuántos días, cuánto es la multa, etc.

FastAPI también genera automáticamente una página de documentación, llamada **Swagger**, donde se pueden ver y probar todos los "pedidos" que acepta la API.

PostgreSQL: la memoria

Una **base de datos** es donde se guarda la información de forma permanente y ordenada, en tablas parecidas a las de Excel pero mucho más potentes y seguras. **PostgreSQL** es una de las bases de datos gratuitas más usadas del mundo. Ahí vivirán los usuarios, los libros, los préstamos y las multas.

Para que Python hable con la base de datos usamos **SQLAlchemy**, que traduce el código Python al lenguaje de la base de datos (SQL).

Docker: la "caja" que funciona igual en todas partes

Un problema clásico en programación es: "*en mi computador sí funciona, pero en el tuyo no*". Pasa porque cada computador tiene versiones distintas de programas instalados.

Docker lo resuelve metiendo cada parte de la aplicación en un **contenedor**. Un contenedor es como una caja sellada que trae adentro todo lo necesario para funcionar: la versión correcta de Python, de Node, de PostgreSQL, etc. Así, la aplicación funciona exactamente igual en cualquier computador que tenga Docker.

- Un **Dockerfile** es la receta para construir una de esas cajas.
- **Docker Compose** es el director de orquesta: un archivo (`docker-compose.yml`) que dice "levanta estas tres cajas, conéctalas entre sí y arráncalas en este orden".

Vercel y Render: dónde vivirá la aplicación en internet

Mientras desarrollamos, la aplicación corre en nuestro computador. Cuando esté lista, debe publicarse en internet para que cualquiera la use:

- **Vercel** es un servicio que publica gratis la parte visual (React).
- **Render** es un servicio que publica gratis el backend (Python) y la base de datos (PostgreSQL).

Ya dejamos los archivos de configuración listos para ambos: `vercel.json` y `render.yaml`.

Git y GitHub: el historial y la nube del código

- **Git** es un programa que guarda el historial de cambios del código, como un "control de versiones" o puntos de guardado en un videojuego.
- **GitHub** es una página web donde se sube ese código para tenerlo respaldado en la nube y compartirlo con otras personas.

3. La arquitectura: cómo está organizado el sistema

La **arquitectura de software** es el plano del edificio: decide cómo se dividen las partes y cómo se comunican.

Analizamos tres opciones:

1. **Microservicios.** Dividir la aplicación en muchos programas pequeños e independientes: uno para préstamos, otro para multas, otro para usuarios, cada uno con su propia base de datos. Es lo que usan empresas como Netflix o Amazon. Lo **descartamos** porque es demasiado complejo para este proyecto, y porque préstamos, reservas y multas están tan relacionados que separarlos solo complicaría las cosas.
2. **Monolito modular.** Un solo programa organizado por temas, con una carpeta "préstamos" que tiene todo lo de préstamos, otra para "reservas", etc. Es una buena alternativa.
3. **Arquitectura por capas (N-Tier).** Es la que **elegimos**. Es la más clara, la más fácil de explicar y la más usada en proyectos académicos y empresariales.

¿Qué es la arquitectura por capas?

Se divide el sistema en **tres capas**, y cada una solo habla con la que tiene al lado. La analogía del restaurante ayuda:

Capa	En el restaurante	En nuestro sistema
Presentación	El comedor y el menú, lo que ve el cliente	La página web en React
Lógica de negocio	La cocina, donde se preparan los platos según las recetas	La API en Python con FastAPI
Datos	La despensa, donde se guardan los ingredientes	La base de datos PostgreSQL

El cliente nunca entra a la despensa. Siempre pide al mesero, el mesero lleva el pedido a la cocina y la cocina saca lo que necesita de la despensa. Esto hace que el sistema sea **ordenado, seguro y fácil de mantener**: si mañana cambiamos el diseño de la página, la cocina y la despensa no se enteran.

Dentro de la "cocina" (el backend) también hay subcapas:

- **routers**: reciben los pedidos de la página web. Son los meseros.
- **schemas**: revisan que los datos lleguen bien escritos; por ejemplo, que un correo tenga "@". Son el control de calidad.
- **services**: aplican las reglas: ¿puede prestar?, ¿tiene multas?, ¿cuántos días le tocan? Son los chefs.
- **models**: representan las tablas de la base de datos. Son el inventario de la despensa.

4. Lo que hicimos, paso a paso

Paso 1: Crear la estructura de carpetas

Primero creamos la "estantería" del proyecto: todas las carpetas y archivos donde irá el código, organizados según la arquitectura. Al principio creamos **165 archivos**, siguiendo un estilo muy profesional pero muy dividido.

Paso 2: Simplificar

Esa cantidad de archivos resultaba abrumadora para entender el proyecto. Los **reducimos a unos 70** sin perder profesionalismo:

- Agrupamos los modelos: en lugar de un archivo por cada tipo de recurso, uno solo (`recurso.py`) para las seis colecciones.
- Eliminamos una capa intermedia que no aportaba mucho en un proyecto de este tamaño.
- Unimos las seis páginas de colecciones en una sola página con pestañas.
- Juntamos los scripts de datos iniciales en un solo archivo (`seed.py`).

La estructura final quedó así:

```
backend/        → la cocina (Python)
  app/
    main.py       → la puerta de entrada de la API
    config.py     → lee la configuración (contraseñas, direcciones)
    database.py   → la conexión con la base de datos
    security.py   → inicio de sesión y permisos por rol
    constants.py  → las reglas fijas: días de préstamo, tarifas de multa
    jobs.py       → tareas automáticas (marcar perdidos, vencer reservas)
    utils.py      → herramientas, como contar días hábiles
    models/       → las tablas
    schemas/      → validación de datos
    services/     → las reglas de negocio
    routers/      → los "meseros" de la API
  seed.py       → carga los datos iniciales (usuarios, libros, etc.)
  tests/        → pruebas automáticas

frontend/       → el comedor (React)
  src/
    pages/        → las pantallas: Login, Préstamos, Multas...
    components/   → piezas reutilizables: menú, tablas, ventanas
    api.js        → el que lleva los pedidos a la cocina

docker-compose.yml → el director de orquesta de Docker
render.yaml        → instrucciones para publicar en Render
README.md          → manual del proyecto
```

Paso 3: Configurar Docker

Escribimos las "recetas" (Dockerfiles) para construir tres contenedores:

1. **sgb_db**: la base de datos PostgreSQL.
2. **sgb_backend**: la API en Python.
3. **sgb_frontend**: la página web en React, servida con **Nginx**, un servidor web muy rápido.

En el `docker-compose.yml` definimos que:

- La base de datos arranca primero, y el backend espera hasta confirmar que está lista.
- El backend se conecta a la base de datos usando el nombre `db`, porque Docker crea una red privada entre los contenedores.
- La página web se abre en `localhost:5173` y la API en `localhost:8000`.
- El backend se reinicia solo cada vez que se guarda un cambio en el código, lo que facilita el desarrollo.

Paso 4: El código mínimo para que todo arranque

Escribimos lo justo para comprobar que las tres capas se comunican:

- En el backend, una ruta de prueba llamada `/api/v1/health` (salud). Cuando alguien la consulta, la API le pregunta a la base de datos "¿estás ahí?" y responde

`{"status":"ok","database":"conectada"}`.

- En el frontend, una pantalla de bienvenida que llama a esa ruta y muestra un recuadro **verde** si todo funciona o **rojo** si algo falla.

Paso 5: Encender Docker y probar

Al ejecutar `docker compose up --build` apareció un error: Docker Desktop, el programa que hace funcionar Docker en Windows, no estaba abierto. Lo localizamos en el computador, lo iniciamos y volvimos a ejecutar el comando.

Docker descargó lo necesario (Python, Node, PostgreSQL), construyó las tres cajas y las encendió. Verificamos que:

- La base de datos estaba saludable.
- La API respondía "ok" y confirmaba la conexión con la base de datos.
- La página web cargaba correctamente.

> **Las tres capas quedaron funcionando y comunicándose entre sí.**

Paso 6: Subir el proyecto a GitHub

Iniciamos Git en la carpeta, guardamos todo en un primer "punto de guardado" (*commit*) y lo subimos al repositorio `github.com/kevin9897/desarrollo_Software_II`.

Un detalle importante de seguridad: el archivo `.env`, que contiene contraseñas y claves, **no se subió**. Está excluido a propósito en el archivo `.gitignore`. En su lugar se subió `.env.example`, una plantilla sin datos sensibles.

Paso 7: ¿Cómo compartirlo con otra persona?

No hace falta enviarle nada especial. Como la configuración de Docker está dentro del código, cualquier persona puede:

1. Instalar Docker Desktop y Git.
2. Descargar el proyecto con `git clone`.
3. Copiar la plantilla `.env.example` como `.env`.
4. Ejecutar `docker compose up --build`.

Tendrá exactamente el mismo entorno. Si va a programar en el proyecto, se le agrega como colaboradora en GitHub.

Lo único que **no** se comparte son los datos de la base de datos, porque cada computador tiene la suya. Para eso crearemos el archivo `seed.py`, que llenará automáticamente cualquier base de datos con los mismos usuarios y recursos.

Paso 8: El diagrama de arquitectura

Por último, hicimos un diagrama en **Mermaid**, un lenguaje para dibujar diagramas escribiendo texto. Muestra las tres capas, sus componentes internos, cómo viaja una petición desde el usuario hasta la base de datos, y dónde se publica cada parte (Vercel, Render o Docker en local).

5. ¿En qué punto estamos y qué sigue?

Lo que ya está listo:

- ✓ Requisitos entendidos y arquitectura definida (N-Tier de 3 capas).
- ✓ Estructura de carpetas profesional y entendible.
- ✓ Docker configurado: base de datos, backend y frontend funcionando y conectados.
- ✓ Configuración lista para publicar en Vercel y Render.
- ✓ Proyecto respaldado en GitHub.
- ✓ Diagrama de arquitectura.

Lo que sigue:

1. **Modelos de la base de datos**: crear las tablas de usuarios, recursos de las seis colecciones, préstamos, reservas, renovaciones y multas.
2. **Datos iniciales (seed.py)**: cargar los 50 estudiantes, 10 profesores, 3 bibliotecarios y todos los libros, revistas, mapas y equipos.
3. **Reglas de negocio (services)**: programar cómo se presta, reserva, renueva y multa.
4. **API (routers)**: exponer esas funciones para que la página web las use.
5. **Tareas automáticas**: marcar recursos como perdidos a los 120 días, vencer reservas y calcular multas.
6. **Pantallas en React**: login, catálogo, préstamos, reservas y multas.
7. **Pruebas** y **publicación** en Vercel y Render.

> En resumen: ya construimos los **cimientos y la estructura del edificio**, comprobamos que la luz, el agua y el gas llegan a cada piso, y guardamos los planos en un lugar seguro. Ahora toca levantar las paredes y amoblar cada habitación.